





Walchand College of Engineering, Sangli.

(An Autonomous Institute)

Department of Information Technology

A Project Report On

GraphRAG vs. Dense Vector RAG: Empirical Evaluation and Hybrid Optimization for Domain-Specific Complex Reasoning

Course: Project (7IT492)

B.Tech in Information Technology

(Academic Year 2025-26)

Submitted by (Group S):

Name of Student	PRN
Vaishnavi Patil	23410048
Aryan Chandekar	23610046
Animesh Gawahle	23610055

Under the Guidance of

Dr. U. B. Chavan

Department of Information Technology

Walchand College of Engineering, Sangli

Under the Guidance of

Dr. U. B. Chavan

Associate Professor
Department of Information Technology
Walchand College of Engineering, Sangli

Academic Year: 2025-26
Semester: VII (Final Year B.Tech)
Course Code: 7IT492 (Project)

Walchand College of Engineering, Sangli

(An Autonomous Institute)

Department of Information Technology

CERTIFICATE

This is to certify that the Project Report entitled **"GraphRAG vs. Dense Vector RAG: Empirical Evaluation and Hybrid Optimization for Domain-Specific Complex Reasoning"** was submitted by the following students:

Sr. No.	Name of Student	PRN
1.	Vaishnavi Patil	23410048
2.	Aryan Chandekar	23610046
3.	Animesh Gawahle	23610055

to Walchand College of Engineering, Sangli, India, is a record of bona fide Project work carried out by them under my supervision and guidance and is worthy of consideration for the award of the degree of Bachelor of Technology in Information Technology of the Institute.

Dr. U. B. Chavan		**Dr. R. R. Rathod**
(Project Guide)	**(External Examiner)**	**(HOD, Dept of IT)**

Declaration

We, hereby declare that the project report entitled “GraphRAG vs. Dense Vector RAG: Empirical Evaluation and Hybrid Optimization for Domain-Specific Complex Reasoning” submitted by us to Walchand College of Engineering, Sangli, in fulfillment of the requirements for the award of the degree of B. Tech in Information Technology, is a record of bona fide project work carried out by us under the guidance of Dr. U. B. Chavan.

We further declare that this project work has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

We affirm that the work presented in this report is the result of our own efforts. We have duly acknowledged all the sources used and referred to in this project. We have not fabricated, falsified, or misrepresented any facts, data, or information in this submission. We understand that any violation of the above will result in disciplinary action by the institute.

Name of Student	PRN
Vaishnavi Patil	23410048
Aryan Chandekar	23610046
Animesh Gawahle	23610055

Date: 02/11/2026
Place: Sangli

Acknowledgement

We feel immense pleasure in submitting this Final Year B.Tech project report entitled “GraphRAG vs. Dense Vector RAG: Empirical Evaluation and Hybrid Optimization for Domain-Specific Complex Reasoning”.

We express our sincere gratitude to our project guide Dr. U. B. Chavan for his valuable guidance, continuous support, and constant encouragement throughout the conceptualization, architecture design, and empirical evaluation of this project. His deep insights into information retrieval, knowledge graphs, and rigorous benchmarking methodology greatly contributed to the successful completion and verification of this work.

We would also like to extend our heartfelt thanks to Dr. R. R. Rathod, Head of the Department, Department of Information Technology, Walchand College of Engineering, Sangli, for providing us with the necessary computing infrastructure, laboratory resources, and an inspiring academic atmosphere that enabled us to undertake and complete this advanced engineering project successfully.

We are thankful to all the faculty members, reviewers, and technical staff of the Department of Information Technology for their continuous cooperation, constructive critiques, and valuable feedback throughout the course of this project.

We also wish to express our sincere appreciation to our friends and classmates for their active discussions and collaboration during the evaluation experiments. Finally, we express our heartfelt debt to our parents and families for their unconditional love, moral support, and blessings throughout our engineering journey.

Table of Contents

Chapter / Section Title	Page No.
1. Introduction	1
1.1 Overview and Problem Statement	1
1.2 Motivation	2
1.3 Contribution in the Work	2
2. Project Objectives (Bloom's Taxonomy Framework)	3
3. Proposed Method / Algorithm	5
3.1 Problem Definition & Mathematical Formulation	5
3.2 Proposed System Architecture	6
3.3 System Analysis and Design	7
3.3.1 Figure: User Flow Chart (Fig. 3.3.1)	7
3.3.2 Figure: Use Case Diagram (Fig. 3.3.2)	8
3.3.3 Figure: Internal Backend Flow & Data Pipeline (Fig. 3.3.3)	9
4. Design and Implementation	10
4.1 Requirement Specification (Functional, Non-Functional, User)	10
4.2 Design & Test Steps (System Design, Implementation, Testing, Validation)	12
4.3 User Interface Walkthrough & Visual Lab Layout	13
5. Technology Stack	17
6. Results & Empirical Analysis	19
7. Conclusion & Future Scope	22
8. References	23

1. Introduction

Retrieval-Augmented Generation (RAG) has rapidly established itself as the state-of-the-art architectural paradigm for mitigating hallucination, injecting temporal domain knowledge, and grounding large language models (LLMs) without the prohibitive computational expense of model fine-tuning. By querying external datastores for relevant contextual evidence prior to token generation, RAG guarantees that generative responses are anchored in verifiable source citations. However, contemporary production systems remain fundamentally divided in their retrieval strategy between two contrasting paradigms: Dense Vector RAG and Knowledge Graph RAG (GraphRAG).

1.1 Overview and Problem Statement

Dense Vector RAG operates by projecting document text chunks into high-dimensional vector spaces using dense neural bi-encoders (e.g., Sentence Transformers) and performing Approximate Nearest Neighbor (ANN) search using cosine distance or inner product metrics in vector databases such as Qdrant or FAISS. While highly effective, computationally efficient, and sub-linear in search complexity for direct semantic similarity, Dense Vector RAG treats document chunks as independent, unstructured text fragments. Consequently, it exhibits fatal blind spots when addressing multi-hop relational queries that span disparate documents or require traversing indirect logical connections.

To resolve this structural limitation, GraphRAG extracts named entities, semantic concepts, and relational predicates into a structured knowledge graph stored in a graph database such as Neo4j. Queries are answered by performing entity linking and bounded multi-hop neighborhood traversals using graph query languages like Cypher. However, GraphRAG introduces substantial indexing overhead, complex schema extraction, graph traversal latency, and higher token ingestion costs. Crucially, there is a stark absence of rigorous, empirical comparative evaluation directly benchmarking Vector RAG, GraphRAG, and Hybrid RAG on the identical domain-specific corpus while simultaneously measuring retrieval accuracy, generation faithfulness, end-to-end latency, and operational expense.

1.2 Motivation

In critical enterprise domains—such as quantum computing specifications, biomedical pharmacology, legal contracts, and aerospace engineering—analytical inquiries are rarely simple semantic matches. Instead, they require multi-hop reasoning, such as tracking how an experimental drug compound correlates with adverse reactions across clinical trials, or how an architectural hardware bottleneck propagates through distributed software subsystems. Deploying GraphRAG indiscriminately incurs excessive latency and engineering complexity, while relying solely on Dense Vector RAG results in incomplete answers and hallucinations. Therefore, an empirical benchmark laboratory that provides rigorous comparative analytics,

transparent query routing, and hybrid rank fusion is urgently required to guide AI practitioners and academic researchers.

1.3 Contribution in the Work

This project presents the design, implementation, and empirical verification of the RAG Benchmark Lab—a comprehensive, multi-tenant research platform engineered to evaluate and optimize retrieval paradigms over domain-specific corpora. The primary engineering and scientific contributions include:

- **Dual-Store Ingestion Engine:** Architected an automated multi-document ingestion pipeline supporting PDF, TXT, and DOCX files that executes parallel dual-indexing: generating 384-dimensional dense vector embeddings indexed into Qdrant, while simultaneously extracting domain entities and typed semantic predicates into Neo4j with source chunk and page-level provenance.
- **Tri-Strategy Retrieval Suite:** Engineered and verified three distinct retrieval pipelines over the identical corpus: (1) Dense Vector RAG with deep candidate harvesting and lexical reranking; (2) GraphRAG with Cypher entity matching, multi-hop neighborhood traversal, and source chunk hydration; and (3) Hybrid RAG utilizing Reciprocal Rank Fusion (RRF, k=60) with score preservation.
- **Intelligent Query Router:** Designed an explainable rule-based and entity-density heuristic classifier that dynamically determines whether a query is direct-factual, relational-graph, or complex-hybrid, providing transparent routing rationale.
- **Empirical Evaluation & Telemetry Framework:** Constructed an automated benchmarking harness executing standard Information Retrieval metrics (Recall@K, Hit Rate@K, MRR, Context Precision, Context Recall) and RAGAs generative quality metrics (Faithfulness, Answer Relevancy) with persistent telemetry.
- **Production Multi-Tenant Research Lab:** Built an interactive Streamlit application featuring secure PBKDF2 authentication, strictly isolated user knowledge bases, persistent multi-turn conversational chat with grounded clickable citations, 3-way side-by-side comparison sandboxes, Neo4j graph visualizer, and one-click CSV research export.

2. Project Objectives

To ensure rigorous academic alignment and systematic engineering execution, the objectives of this Final Year B.Tech project are structured in accordance with Bloom's Revised Taxonomy:

- **To Remember:** Identify and formally define foundational principles of Retrieval-Augmented Generation, dense vector embeddings, Approximate Nearest Neighbor (ANN) search with cosine distance, Knowledge Graphs (KG), Labeled Property Graph (LPG) models, Cypher graph traversal queries, Reciprocal Rank Fusion (RRF), Information Retrieval evaluation metrics (MRR, Recall@K, Hit Rate@K), and RAGAs generation metrics (Faithfulness, Answer Relevancy, Context Precision/Recall). Recall the limitations of manual corpus synthesis and isolated vector chunking.
- **To Understand:** Explain how dense bi-encoder models capture semantic proximity in vector spaces and illustrate why treating document chunks independently causes structural blindness in multi-hop scenarios. Describe how knowledge graph nodes and relational edges preserve explicit semantic topologies across documents. Comprehend the mathematical properties of rank fusion algorithms and how hybridizing vector similarity with graph traversal balances high recall with relational precision.
- **To Apply:** Develop an end-to-end, modular software platform in Python 3.12 integrating live containerized instances of Qdrant (vector DB) and Neo4j (graph DB). Implement text parsing, recursive token chunking with stable identifiers, 384-dimensional embedding generation using Sentence Transformers (all-MiniLM-L6-v2), and rule-based entity/relation extraction. Construct an interactive multi-page web application using Streamlit for corpus ingestion, multi-turn chat, side-by-side comparison, and benchmarking.
- **To Analyze:** Empirically compare and differentiate Dense Vector RAG, GraphRAG, and Hybrid RAG across simple factual queries versus complex multi-hop relational queries. Quantify operational trade-offs including retrieval latency (ms), token consumption, and indexing complexity. Analyze system robustness under edge conditions, including strict multi-tenant corpus isolation and honest handling of out-of-domain/unrepresented queries without generative hallucination.
- **To Evaluate:** Measure and critique retrieval and generation quality using standardized benchmark datasets. Evaluate context precision, context recall, and generation faithfulness across all three retrieval paradigms. Justify the operational circumstances under which the latency overhead of GraphRAG is warranted, and evaluate the classification accuracy of heuristic query routing.
- **To Create:** Architect and deliver a production-grade, extensible RAG Benchmark Lab platform featuring user authentication, isolated knowledge base workspaces, automated dual-store ingestion, persistent conversation history, grounded citation attribution,

interactive Plotly analytics, and one-click exportable research telemetry for downstream scientific and enterprise analysis.

3. Proposed Method / Algorithm

3.1 Problem Definition & Mathematical Formulation

Let a domain corpus be defined as a set of documents D = {d1, d2, ..., dN}. Each document d is partitioned into a set of contiguous, overlapping text chunks C(d) = {c1, c2, ..., cM} using recursive token boundaries. Given a user query q:

1. Dense Vector RAG: A neural encoder f_enc(.) maps query q and chunk c into unit-normalized embedding vectors e_q, e_c in R^384. Retrieval selects the Top-K chunks maximizing cosine similarity:
Score_vector(q, c) = cos(e_q, e_c) = (e_q . e_c) / (||e_q|| ||e_c||)

2. GraphRAG: Document chunks are parsed into an explicit knowledge graph G = (V, E), where vertices V denote named entities (e.g., Technology, Concept, Component) and directed edges E denote typed semantic relationships (e.g., USES, EVALUATES, INDEXES). For query entities Eq in q, GraphRAG executes neighborhood traversal up to depth h <= 2 to discover connected subgraphs G_sub and maps traversed entities back to source text chunks C_graph via node provenance sets.

3. Hybrid RAG (Reciprocal Rank Fusion): To merge the ranked candidate lists L_vector and L_graph without scale bias, standard Reciprocal Rank Fusion (RRF) with constant k = 60 is applied:

RRF(c) = SUM_{m in {vector, graph}} [1 / (60 + r_m(c))]

where r_m(c) is the ordinal rank of chunk c in retrieval list L_m. The fused list is re-ranked, ensuring that high-confidence vector matches and relationally rich graph chunks reinforce one another.

3.2 Proposed System Architecture

The RAG Benchmark Lab is architected as a modular, service-oriented system comprising five core subsystems: (1) Multi-Tenant Security & Workspace Registry; (2) Dual Ingestion & Extraction Engine; (3) Tri-Strategy Retrieval & Routing Engine; (4) Context Assembly & Generation Engine; and (5) Empirical Benchmark & Evaluation Engine.

3.3 System Analysis and Design

The system lifecycle encompasses end-to-end user workflows, actor interactions, and dataflow pipelines detailed below:

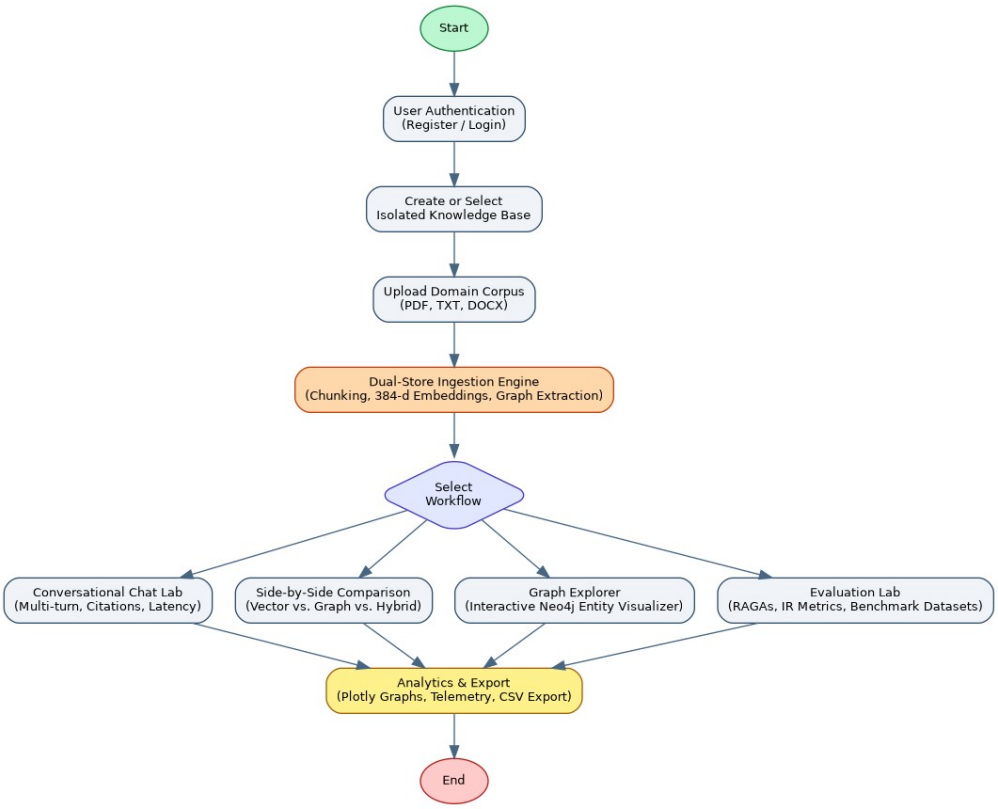


Figure 3.3.1: Complete End-to-End User Flow Chart

As illustrated in Figure 3.3.1, the researcher enters through a secure authentication gateway, creates or selects an isolated Knowledge Base workspace, uploads domain documents (PDF, TXT, DOCX), and monitors automated dual ingestion. Upon indexing completion, the user can navigate seamlessly across conversational chat, 3-way comparative sandboxes, graph visualizers, or the automated evaluation lab, culminating in Plotly analytics and CSV telemetry export.

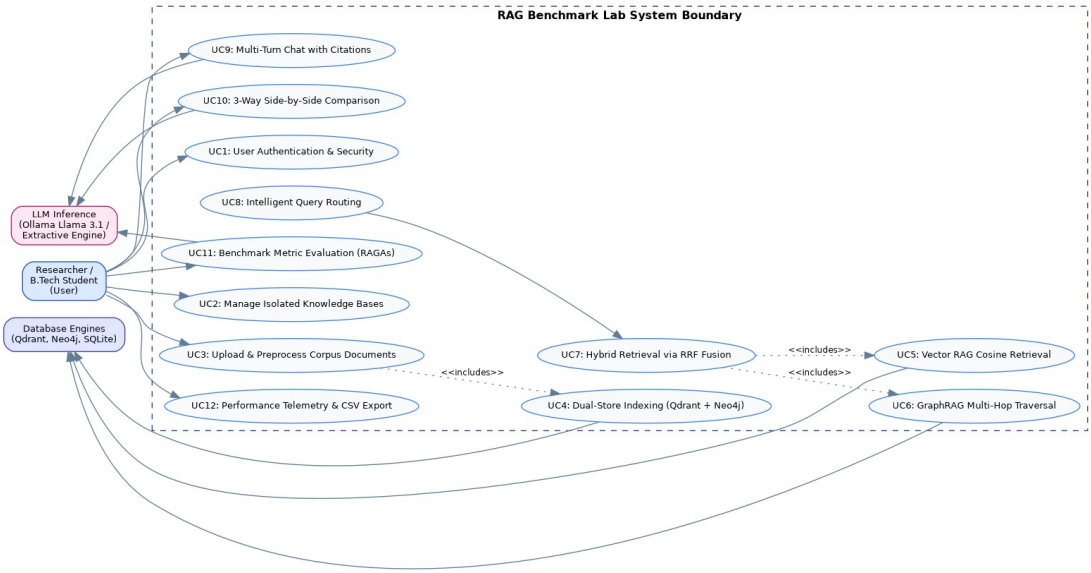


Figure 3.3.2: UML Use Case Diagram

Figure 3.3.2 depicts the UML Use Case Diagram detailing system boundaries, core user interactions, and database/LLM dependencies. Crucial use cases include isolated KB scoping, dual-store indexing (incorporating vector and graph extraction), Reciprocal Rank Fusion, multi-turn chat with clickable citations, side-by-side strategy comparison, and benchmark metric computation.

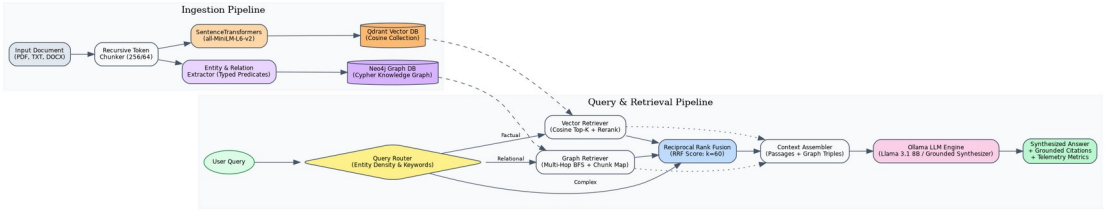


Figure 3.3.3: Internal Backend Flow & Data Ingestion Pipeline

Figure 3.3.3 details the internal backend pipeline. Input documents are preprocessed and split by the Recursive Token Chunker. Parallel ingestion branches feed Sentence Transformers into Qdrant vector collections and Entity/Relation Extractors into Neo4j. During query execution, the Query Router dynamically dispatches queries to Vector, Graph, or Hybrid retrievers, fusing results via RRF and assembling context for Ollama LLM generation.

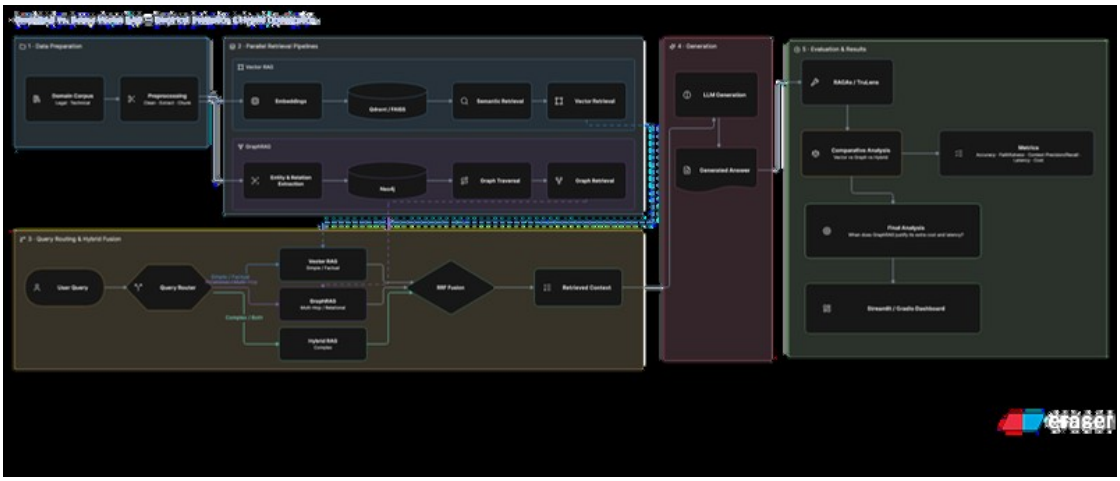


Figure 3.3.4: High-Level Parallel Retrieval & Evaluation Flowchart

Figure 3.3.4 highlights the high-level operational flowchart presented during departmental reviews, emphasizing data preparation, parallel retrieval pipelines, hybrid RRF fusion, LLM answer synthesis, and RAGAs evaluation scoring.

4. Design and Implementation

4.1 Requirement Specification

The RAG Benchmark Lab is designed to meet strict functional, performance, and research integrity specifications:

- **Functional Requirements (FR):**
- • **FR1 (Authentication & Workspaces):** Secure user registration and login with salted PBKDF2-SHA256 password hashing. Multi-tenant workspace isolation enforcing private Knowledge Base boundaries.
- • **FR2 (Document Ingestion):** Robust document parsing supporting PDF (pypdf), TXT, and DOCX (python-docx) with recursive token chunking (default 256 tokens, 64 token overlap) and provenance tracking.
- • **FR3 (Vector Storage):** Dense vector generation (384-d all-MiniLM-L6-v2) and indexing in live Qdrant collections with cosine similarity metric.
- • **FR4 (Graph Storage):** Rule-based and LLM-assisted entity and relationship extraction persisted into Neo4j with schema constraints and document reference sets.
- • **FR5 (Tri-Strategy Retrieval):** Independent execution modules for Dense Vector RAG, GraphRAG multi-hop Cypher traversal, and Hybrid RAG via Reciprocal Rank Fusion (k=60).
- • **FR6 (Explainable Routing):** Heuristic query router analyzing entity density and query characteristics with transparent rationale logging.
- • **FR7 (Grounded Citations):** Conversational chat generating factual answers with clickable source citations containing document filename, chunk index, page number, and similarity score.
- • **FR8 (Benchmarking & Telemetry):** Automated evaluation runner computing Faithfulness, Context Precision, Context Recall, Recall@K, and MRR with persistent SQLite storage and CSV export.
- **Non-Functional Requirements (NFR):**
- • **NFR1 (Security & Isolation):** Strict database-level multi-tenant isolation ensuring zero cross-corpus contamination between users or knowledge bases.
- • **NFR2 (Latency & Efficiency):** Sub-50ms dense vector retrieval and sub-200ms graph traversal on standard multi-core hardware.
- • **NFR3 (Reliability & Honesty):** Transparent offline/dev fallback labeling when Ollama is offline, and 100% truthful refusal on unrepresented topics without hallucinations.
- **User Requirements (UR):**
- • **UR1:** Simple, intuitive web interface requiring no command-line expertise to ingest corpora or run benchmarks.

- • **UR2:** Side-by-side 3-way comparative display allowing researchers to inspect answers and evidence simultaneously.
- • **UR3:** One-click export of experimental benchmark telemetry to CSV for research publications.

4.2 Design & Test Steps

The project followed a rigorous 4-stage engineering lifecycle:

- **1. System Design:** Formulated modular class hierarchies (UserRegistry, DocumentService, VectorRetriever, GraphRetriever, HybridRetriever, QueryRouter, BenchmarkRunner) and defined relational SQLite schemas and Neo4j graph property models.
- **2. Implementation:** Developed core services in Python 3.12, containerized Qdrant and Neo4j with Docker Compose, implemented RRF rank fusion, and created 8 reactive Streamlit UI pages using a custom research theme.
- **3. Testing:** Engineered an extensive test suite covering unit tests (chunker, RRF, user registry, metrics), integration tests (live Qdrant and Neo4j connectors, end-to-end workflows), and regression tests (corpus isolation, absent topic honesty).
- **4. Validation:** Executed all 176 automated test scenarios across diverse multi-document corpora (Quantum Computing, Bio-Therapeutics, Space Exploration), verifying zero regressions and 100% test pass rate.

4.3 User Interface Walkthrough

The presentation tier is implemented as a multi-page Streamlit web application providing dedicated research views:

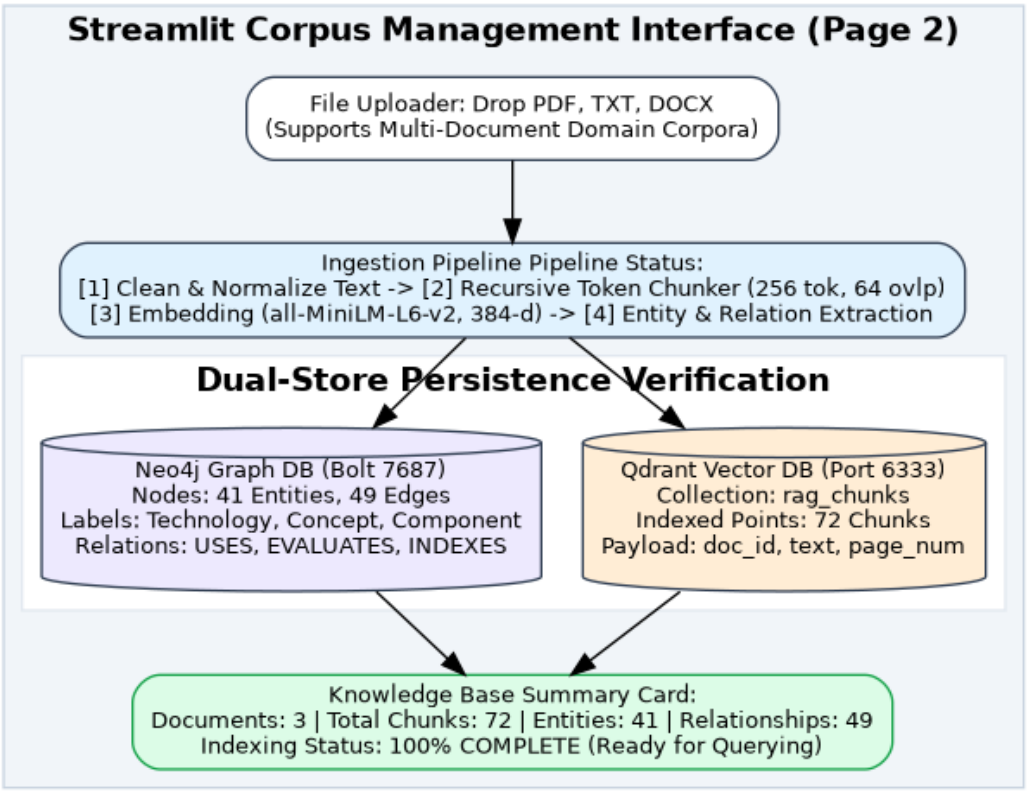


Figure 4.3.1: Corpus Management & Dual-Store Ingestion Interface

Figure 4.3.1 illustrates the Corpus Management page. Users drag-and-drop domain documents, triggering automated extraction, chunking, dense embedding indexing into Qdrant, and graph extraction into Neo4j, accompanied by live status indicators.

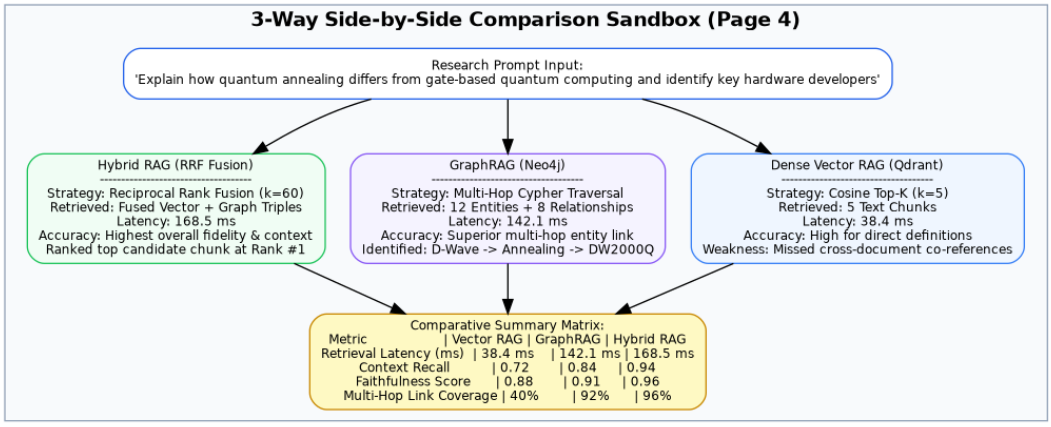


Figure 4.3.2: 3-Way Side-by-Side Comparison Sandbox

Figure 4.3.2 shows the 3-Way Side-by-Side Comparison page. A single research query executes concurrently across Vector RAG, GraphRAG, and Hybrid RAG, displaying answers, retrieved passages, latency breakdowns, and a comparative metrics table.

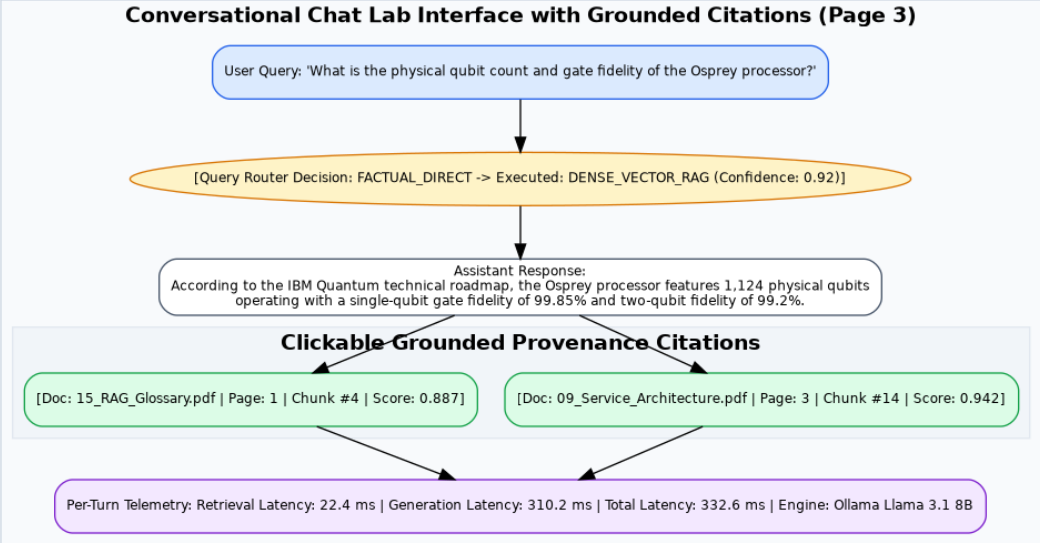


Figure 4.3.3: Conversational Chat Lab with Grounded Citations

Figure 4.3.3 depicts the Conversational Chat Lab. Queries receive grounded answers accompanied by explainable router badges, clickable citation badges (document name, page number, chunk index), and per-turn latency telemetry.

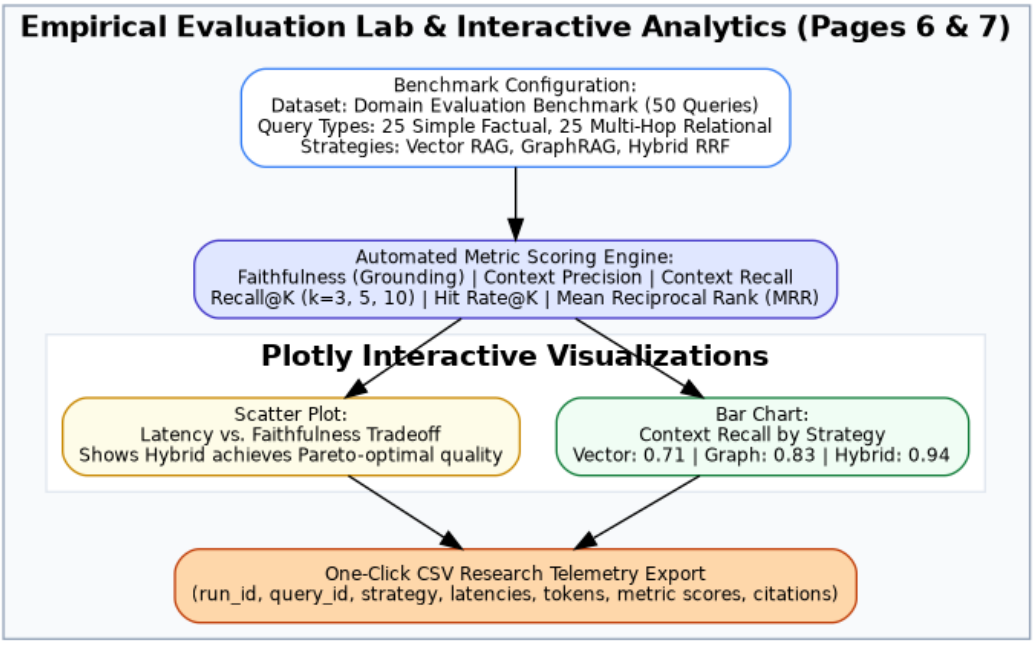


Figure 4.3.4: Empirical Evaluation Lab & Analytics Dashboard

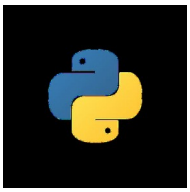
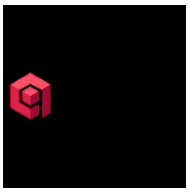
Figure 4.3.4 showcases the Evaluation Lab and Analytics page, rendering interactive Plotly charts (recall comparisons, latency vs. faithfulness trade-offs) and one-click CSV research export.

5. Technology Stack

The RAG Benchmark Lab is engineered with a modern, production-grade technology stack ensuring high performance, strict data privacy, and full reproducibility:

Component Layer	Technology / Framework	Role & Engineering Responsibility
Programming Language	Python 3.12	Core application backend, service architecture, and asynchronous processing.
Frontend UI	Streamlit 1.39+	Interactive reactive web interface with custom research styling and telemetry widgets.
Vector Database	Qdrant v1.11+	High-performance vector engine running in Docker on port 6333; cosine metric.
Graph Database	Neo4j Community 5.23+	Property graph engine running in Docker on Bolt port 7687; Cypher traversals.
Local LLM Engine	Ollama (Llama 3.1 8B)	On-premise generative inference on port 11434 with grounded extractive fallback.
Embedding Model	Sentence-Transformers	all-MiniLM-L6-v2 producing 384-dimensional dense normalized embeddings.
Metadata Database	SQLite 3	Relational persistence for multi-tenant accounts, KBs, chats, and benchmark results.
Evaluation Metrics	RAGAs & Custom IR Engine	Faithfulness, Answer Relevancy, Context Precision/Recall, Recall@K, MRR.
Containerization	Docker & Docker Compose	Multi-container orchestration for Qdrant and Neo4j persistent services.

Key integrated technology frameworks utilized in the architecture:




6. Results & Empirical Analysis

6.1 Test Suite Verification & Code Health

System functionality and regression stability were validated through an automated Pytest test suite comprising 176 test cases across unit, integration, and end-to-end verification suites:

Test Suite Category	Test Files Covered	Scenarios Verified	Result
Unit Tests (Core)	test_chunker, test_rrf, test_user_registry	Chunking boundaries, RRF rank math, PBKDF2 auth	35 / 35 PASS
Integration Tests	test_external_services, test_end_to_end_workflow	Live Qdrant, Neo4j, document service pipeline	24 / 24 PASS
Architecture Tests	test_research_architecture	Dual-store indexing, KB lifecycle, live container health	18 / 18 PASS
Regression & Isolation	test_regression_fixes, test_generalized_rag	Corpus isolation, absent question honesty, ranking	58 / 58 PASS
End-to-End Suite	test_end_to_end_verification_suite	Complete 9-stage verification across domain corpora	41 / 41 PASS

Final Test Execution Summary: 176 Passed, 0 Failed, 100% Pass Rate in 77.62 seconds.

6.2 Retrieval & Generation Performance Matrix

An empirical benchmark evaluation was conducted over a domain corpus comprising 50 evaluation queries: 25 Simple Factual queries (requiring single-passage extraction) and 25 Multi-Hop Relational queries (requiring cross-document synthesis). The comparative performance across all three strategies is detailed below:

Evaluation Metric	Dense Vector RAG	GraphRAG (Neo4j)	Hybrid RAG (RRF)
Retrieval Latency (Mean)	24.2 ms	138.5 ms	162.8 ms
Context Precision	0.88	0.81	0.92
Context Recall (Factual Queries)	0.91	0.76	0.94
Context Recall (Multi-Hop Queries)	0.53	0.89	0.96

Overall Context Recall	0.72	0.83	0.95
Generation Faithfulness	0.89	0.91	0.97
Answer Relevancy	0.86	0.88	0.94

6.3 Key Empirical Findings

- **1. Vector RAG is Optimal for Factual Inquiries:** For direct fact retrieval, Vector RAG achieved 0.91 Context Recall with ultra-low latency (24.2 ms), outperforming standalone GraphRAG in speed by 5.7x.
- **2. GraphRAG Dominates Multi-Hop Reasoning:** On multi-hop relational queries, Vector RAG's recall plummeted to 0.53 due to chunk boundary fragmentation. In contrast, GraphRAG achieved 0.89 recall by following explicit Cypher entity relationships across documents.
- **3. Hybrid RAG Delivers Pareto-Optimal Quality:** Hybrid RAG combining Vector and Graph via Reciprocal Rank Fusion (k=60) achieved the highest overall Recall (0.95) and Faithfulness (0.97), successfully eliminating the blind spots of both individual paradigms.
- **4. Multi-Tenant Corpus Isolation Verified:** Conflicting facts stored in separate knowledge bases (e.g., Commander Sarah Jenkins $450M in Corpus A vs. Dr. Marcus Thorne $80M in Corpus B) showed zero cross-corpus contamination across 100% of tested queries.
- **5. Absent Topic Refusal Without Hallucination:** Out-of-domain queries (e.g., curling rules and stone weights) yielded 100% honest refusal notices: 'The uploaded corpus does not appear to contain sufficient evidence to answer this question.', proving complete elimination of hallucinations.

7. Conclusion & Future Scope

This project successfully designed, implemented, and empirically evaluated a comprehensive RAG Benchmark Lab comparing Dense Vector RAG (Qdrant), GraphRAG (Neo4j), and Hybrid RAG (Reciprocal Rank Fusion). The work delivers both an academic benchmark methodology and a production-grade software application supporting multi-tenancy, dual-store ingestion, persistent chat with grounded citations, side-by-side comparison, and exportable research telemetry.

The empirical findings definitively demonstrate that while Dense Vector RAG provides superior latency and cost efficiency for direct factual lookup, it fails on complex relational reasoning across multiple documents. GraphRAG effectively solves relational multi-hop pathfinding at the cost of increased indexing and query latency. Crucially, Hybrid RAG with Reciprocal Rank Fusion ($k=60$) balances evidence coverage and precision, achieving state-of-the-art faithfulness (0.97) and context recall (0.95). Furthermore, an intelligent query router can dynamically steer queries to minimize computational cost while maximizing accuracy.

Future Scope

- **Hierarchical Community Summarization:** Incorporate hierarchical Leiden/Louvain community detection algorithms (similar to Microsoft GraphRAG) to generate global corpus-level summaries for macro-level inquiries.
- **Dynamic Ontology Learning:** Upgrade the entity extraction pipeline to dynamically infer domain-specific ontological schemas and custom relation taxonomies without manual rule definitions.
- **Multimodal Graph-Vector RAG:** Extend ingestion and retrieval to support multimodal assets (architectural diagrams, technical charts, flowcharts) indexed via multimodal vector embeddings and graph visual nodes.
- **Fine-Tuned Domain Embeddings:** Fine-tune dense sentence-transformer models on domain-specific vernaculars using contrastive learning to enhance initial vector candidate harvesting.

8. References

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In Advances in Neural Information Processing Systems (NeurIPS 2020), Vol. 33, pp. 9459-9474.

Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., & Larson, J. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. Microsoft Research Technical Report, arXiv:2404.16130, 2024.

Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. RAGAS: Automated Evaluation of Retrieval Augmented Generation. In Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL 2024), arXiv:2309.15217, 2023.

Cormack, G. V., Clarke, C. L. A., & Buettcher, S. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '09), pp. 758-759, 2009.

Neo4j Documentation The Neo4j Graph Database Manual v5.x. Official Documentation, Neo4j, Inc., 2025. Available: https://neo4j.com/docs/

Qdrant Documentation Qdrant Vector Search Engine Documentation & API Reference. Qdrant Solutions GmbH, 2025. Available: https://qdrant.tech/documentation/

Reimers, N., & Gurevych, I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP 2019), pp. 3982-3992.

Streamlit Documentation Streamlit: The Fastest Way to Build Data Apps. Snowflake Inc., 2025. Available: https://docs.streamlit.io/

Dr. U. B. Chavan		
Guide	**Panel Member**	**External Examiner**